



S-OS

内核设计汇报

S-OS成员：包一帆、李嘉乐、庄廷钊
指导教师：蔡朝晖



武汉大学
WUHAN UNIVERSITY

01

概要介绍

S-OS (SuperOS) 是一个由C语言编写的宏内核操作系统，面向 RISC-V 64 和 LoongArch 64 两种体系结构，覆盖进程与线程管理、虚拟内存管理（Buddy + Slab + 页表 + VMA）、EXT4 文件系统（集成 lwext4）、VirtIO 块设备驱动（MMIO 与 PCI）以及 Linux 风格系统调用接口（进程、内存、文件、定时器、事件、socket 类别），在初赛适配了 basic、buxybox、libctest、libcbench、lua、部分 ltp 等测试用例，在决赛一阶段通过 cagent 全部测例和 buildstorm 环境检查与最小构建，最终尝试下板。

目录	职责	代表性文件
hal/	架构入口、寄存器现场保存恢复、上下文切换、UART	entry.S, trampoline.S, swtch.S, uart.c
hsai/	Trap 分发、页表硬件配置、PLIC、内核打印	hsai_trap.c, hsai_mem.c, plic.c, print.c
kernel/	进程线程、内存管理、系统调用、信号、定时器	process.c, vmem.c, syscall.c, signal.c, futex.c
kernel/fs/	文件对象、路径处理、块缓存、EXT4 适配	vfs_ext4.c, file.c, bio.c, blockdev.c
kernel/drive r/	块设备驱动（VirtIO MMIO / PCI）	virt.c, pci.c, virtio_disk.c
kernel/exec.c	ELF 与脚本装载	exec.c
user/	首个用户态程序、系统调用桩、测试框架	user.c, usys.S, usys_macro.inc
include/	内核核心头文件、HAL/HSAI 接口、文件系统定义	process.h, thread.h, vmem.h, vma.h

核心亮点

- 01 进程-线程-调度器三层分离的任务模型
- 02 HAL/HSAI实现架构无关性
- 03 Buddy→SLAB→页表→VMA 四层协同的内存管理

Merge branch 'master' into 'main' S-OS队

再次解决ltp父子进程不同步 Developer

完善文档 Developer

Merge branch 'master' into 'main' S-OS队

ltp去除busybox timeout包裹 Developer

Merge branch 'master' into 'main' S-OS队

添加时钟标定&修复父子进程不同步 Developer

Merge branch 'master' into 'main' S-OS队

解决父子进程不同步的问题 Developer

Merge branch 'master' into 'main' S-OS队

增大NPROC到512&注释sleep 5 Developer

Merge branch 'master' into 'main' S-OS队

fix Developer

Merge branch 'master' into 'main' S-OS队

大幅回退修改&调整ltp测例 Developer

Merge branch 'master' into 'main' S-OS队

更改ltp测试顺序 Developer

Merge branch 'master' into 'main' S-OS队

清除无用调试输出 Developer

166 Commits

4 Branches

庄廷钊 管理员

看过了，前面几个没有输出的都是这样

李嘉乐 管理员

庄廷钊 不

tst_test.c:1421: TFAIL: Main test p...

@刁民 是报这个错之后之后的测试点都不输出summary还是这个测试点本身不输出

群主 包一帆

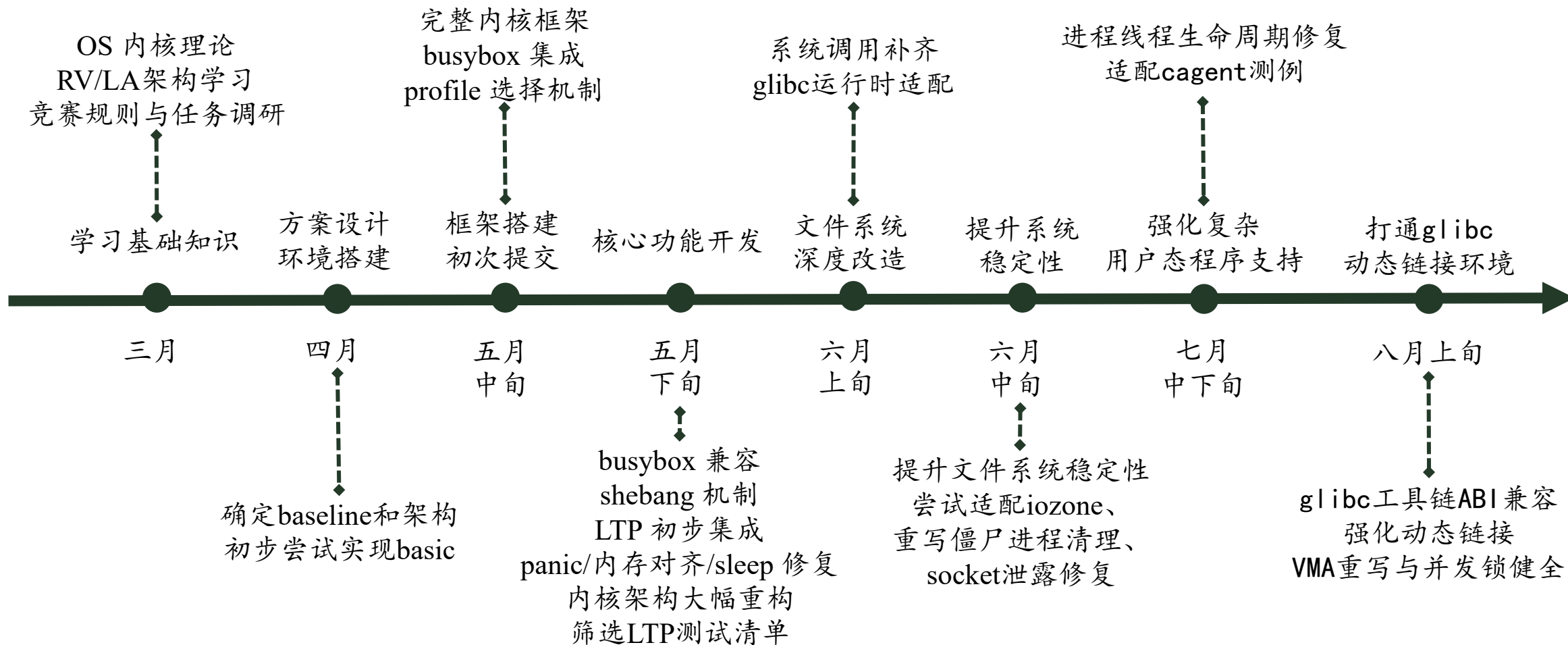
可能是我加的cleanup

庄廷钊 管理员

不输出Summary



开发周期





整体架构

本系统采用解耦的分层架构以实现跨架构（RISC-V/LoongArch）支持：

体系结构无关层：

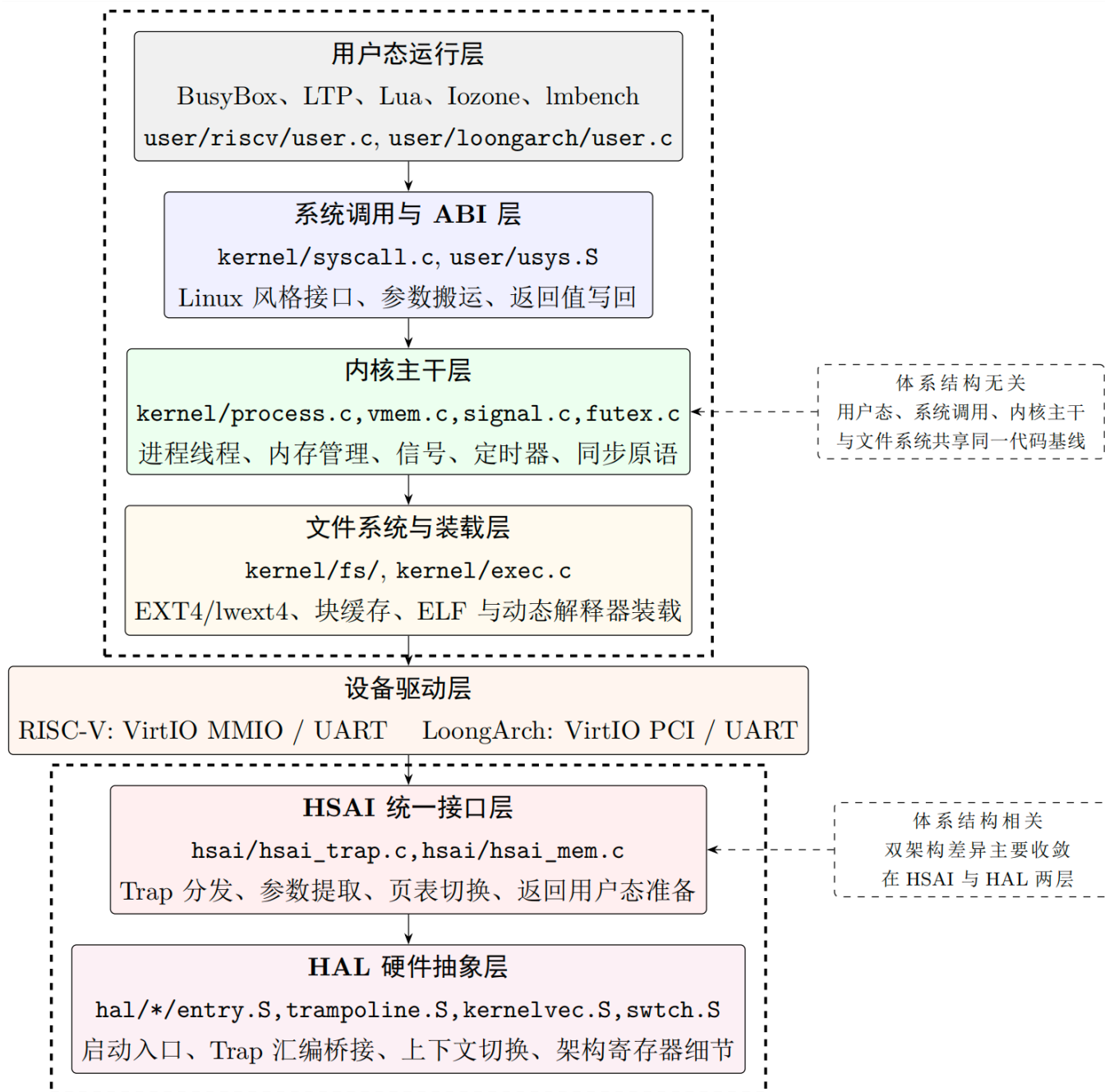
- 涵盖用户态、系统调用、内核主干及文件系统，实现内核核心功能。

设备驱动层：

- 针对不同架构分别适配对应的总线（MMIO/PCI）与外设驱动。

体系结构相关层：

- 通过 HSAI（统一接口层）与 HAL（硬件抽象层）实现 Trap 处理、页表切换及底层寄存器控制。





武汉大学
WUHAN UNIVERSITY

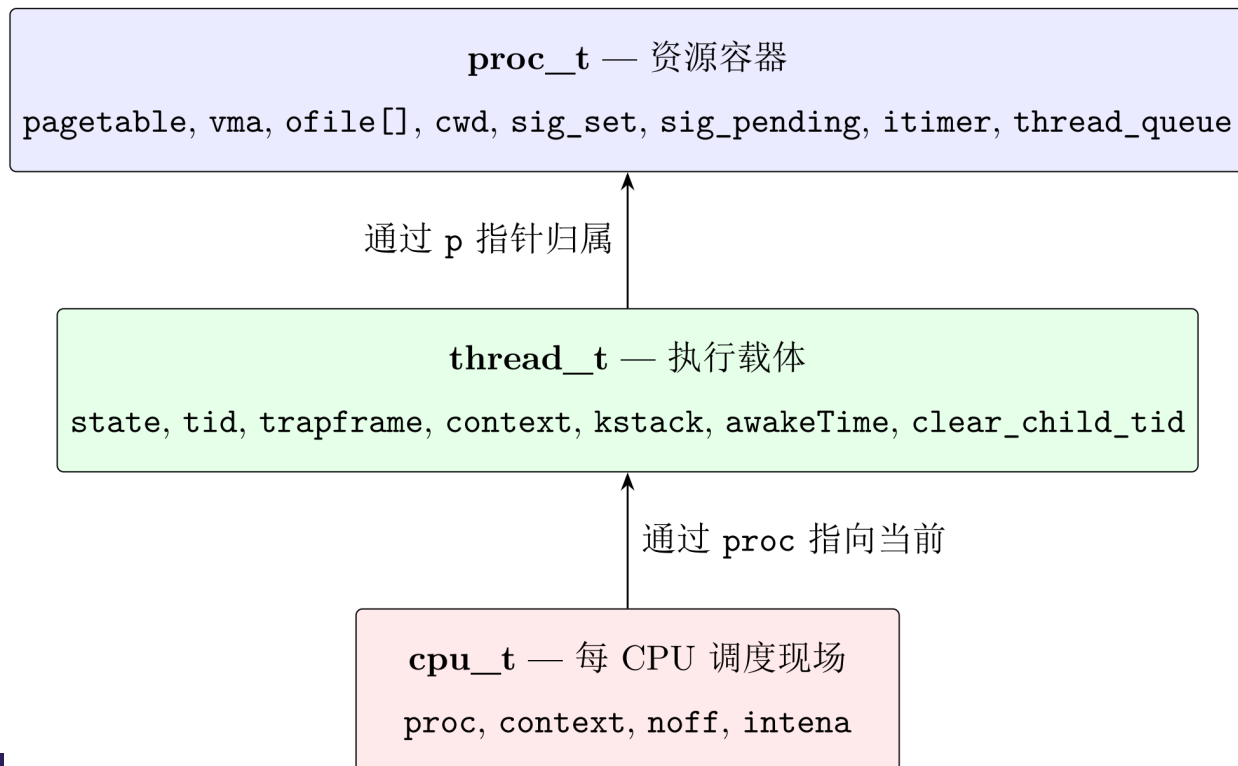
02

核心功能介绍

进程线程管理



武汉大学
WUHAN UNIVERSITY



01 三层任务模型

- proc_t 处理资源;
- thread_t 处理执行现场;
- cpu_t 处理调度上下文

02 多路径创建机制:

- fork 复制进程;
- clone 扩展创建语义;
- clone_thread 共享地址空间建线程。

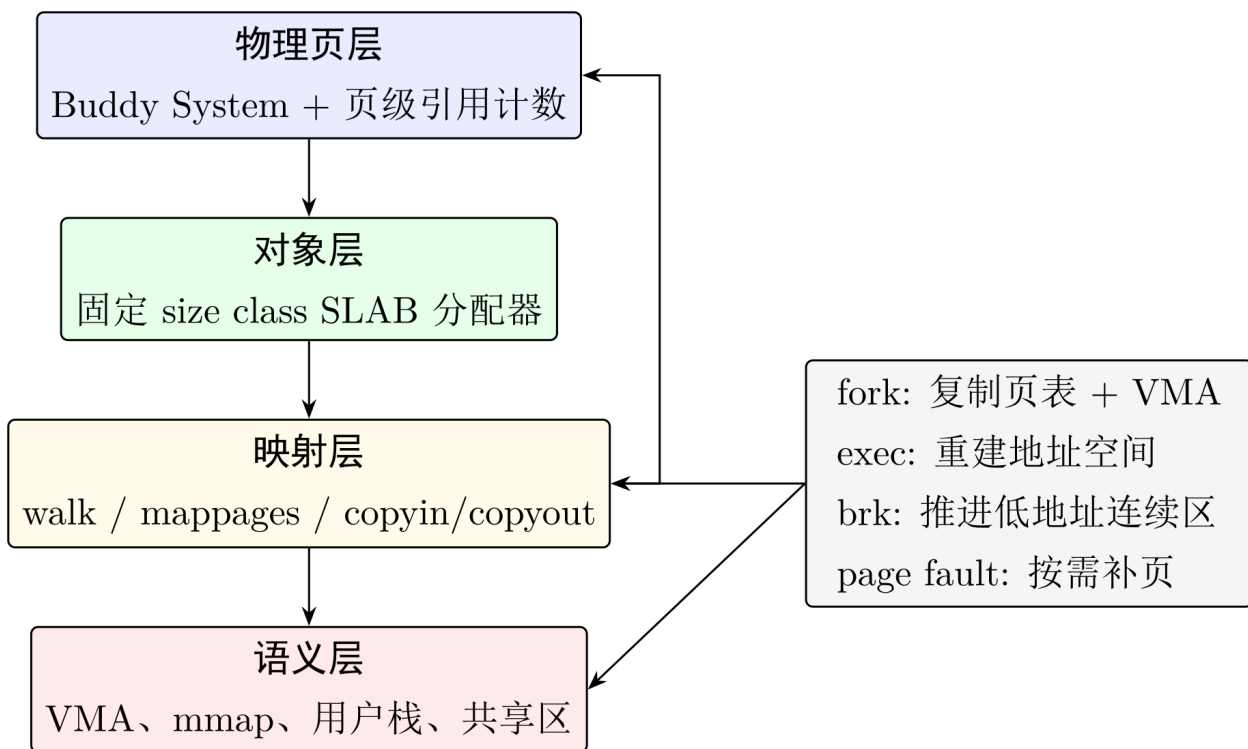
03 调度与同步主链:

- 先选进程再选线程;
- yield/sched 统一回退;
- channel/futex 分层阻塞唤醒。

04 进程生命周期闭环:

- 信号与定时器驱动异步事件;
- exit/wait/freeproc 完成退出与回收。

内存管理



01 四层抽象架构

- Buddy 处理物理页;
- SLAB 处理小对象;
- 页表层处理映射;
- VMA 处理地址空间语义。

02 页表与空间组织

- walk() 兼容双架构;
- 低地址连续区、mmap 区、用户栈分层布局;
- 顶部保留 TRAPFRAME/TRAMPOLINE。

03 映射与共享机制

- mmap/munmap 管理区域;
- MAP_SHARED 配合引用计数;
- fork 区分共享页与私有页复制。

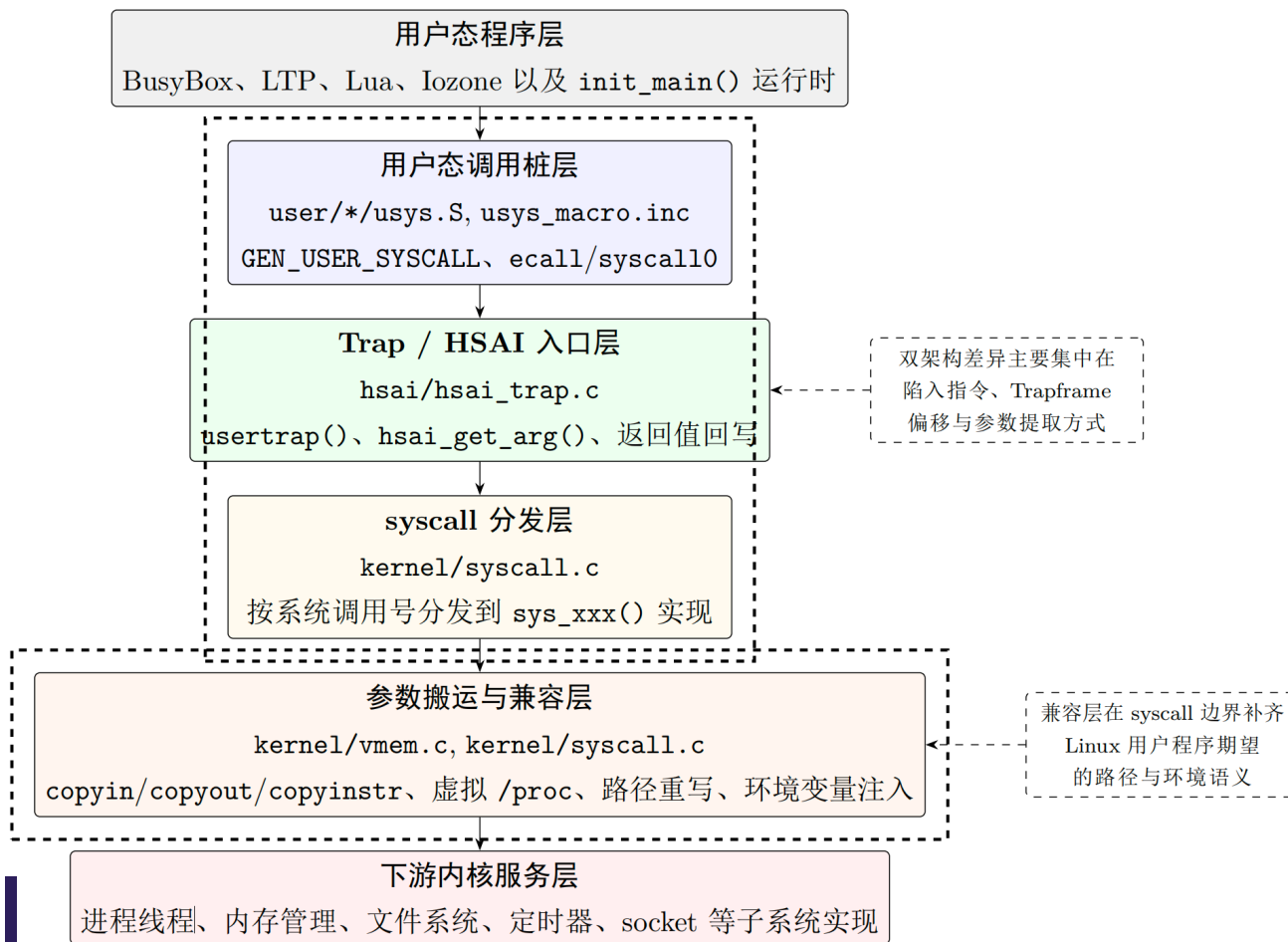
04 运行期内存闭环

- 缺页异常按需补页;
- exec 重建地址空间;
- brk 推进堆边界。

系统调用



武汉大学
WUHAN UNIVERSITY



01 统一 ABI 入口

- 采用 Linux 风格 syscall 编号；
- 双架构调用桩语义一致；
- 通过 ecall/syscall 0 陷入内核。

02 统一分发路径

- usertrap 识别系统调用；
- 提取 a0-a7 参数；
- syscall() 分发到具体 sys_xxx。

03 参数规范化

- copyin/copyout 处理缓冲区；
- copyinstr 处理字符串；
- fetchaddr/fetchstr 处理指针参数。

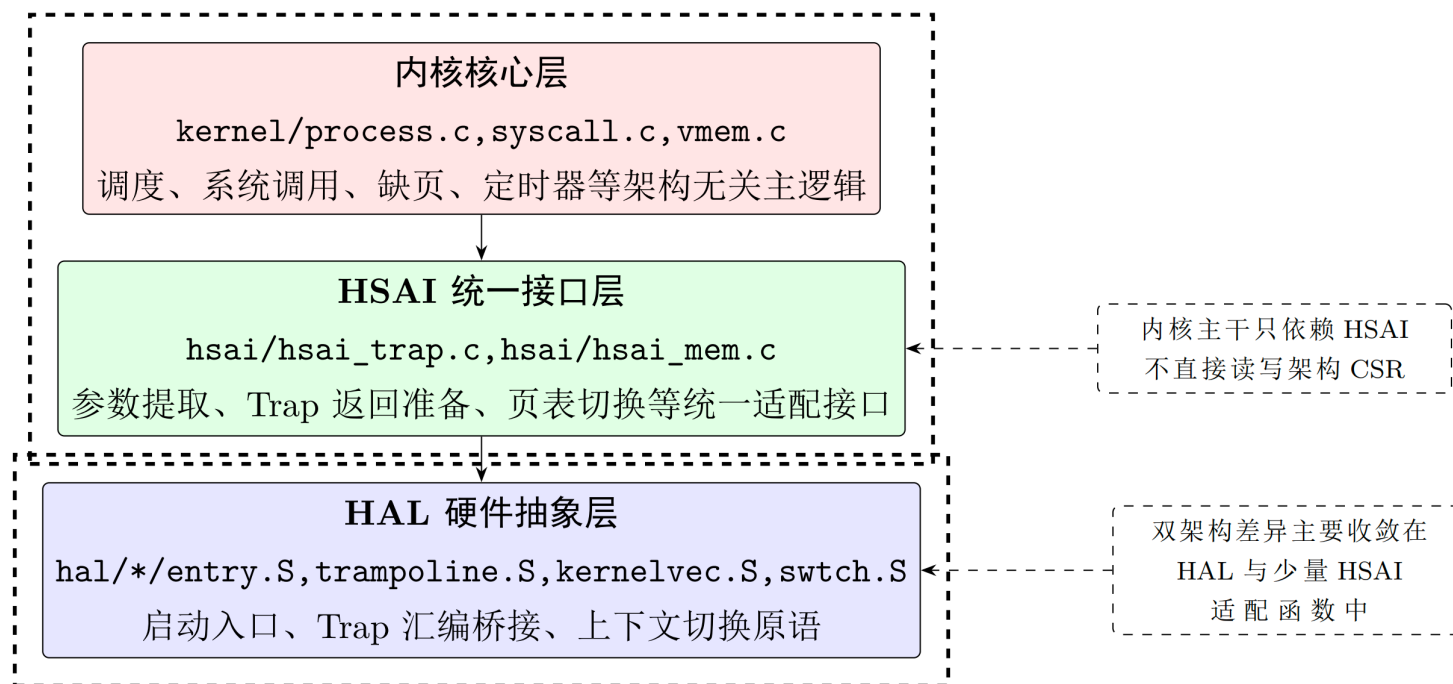
04 兼容层设计

- 虚拟 /proc 文件；
- LD_LIBRARY_PATH 注入；
- /dev/shm 重映射；
- synthetic mount 支撑测试程序。

硬件抽象与异常处理



武汉大学
WUHAN UNIVERSITY



01 分层抽象设计

- HAL 处理底层硬件;
- HSAI 提供统一接口;
- Kernel 保持架构无关主干。

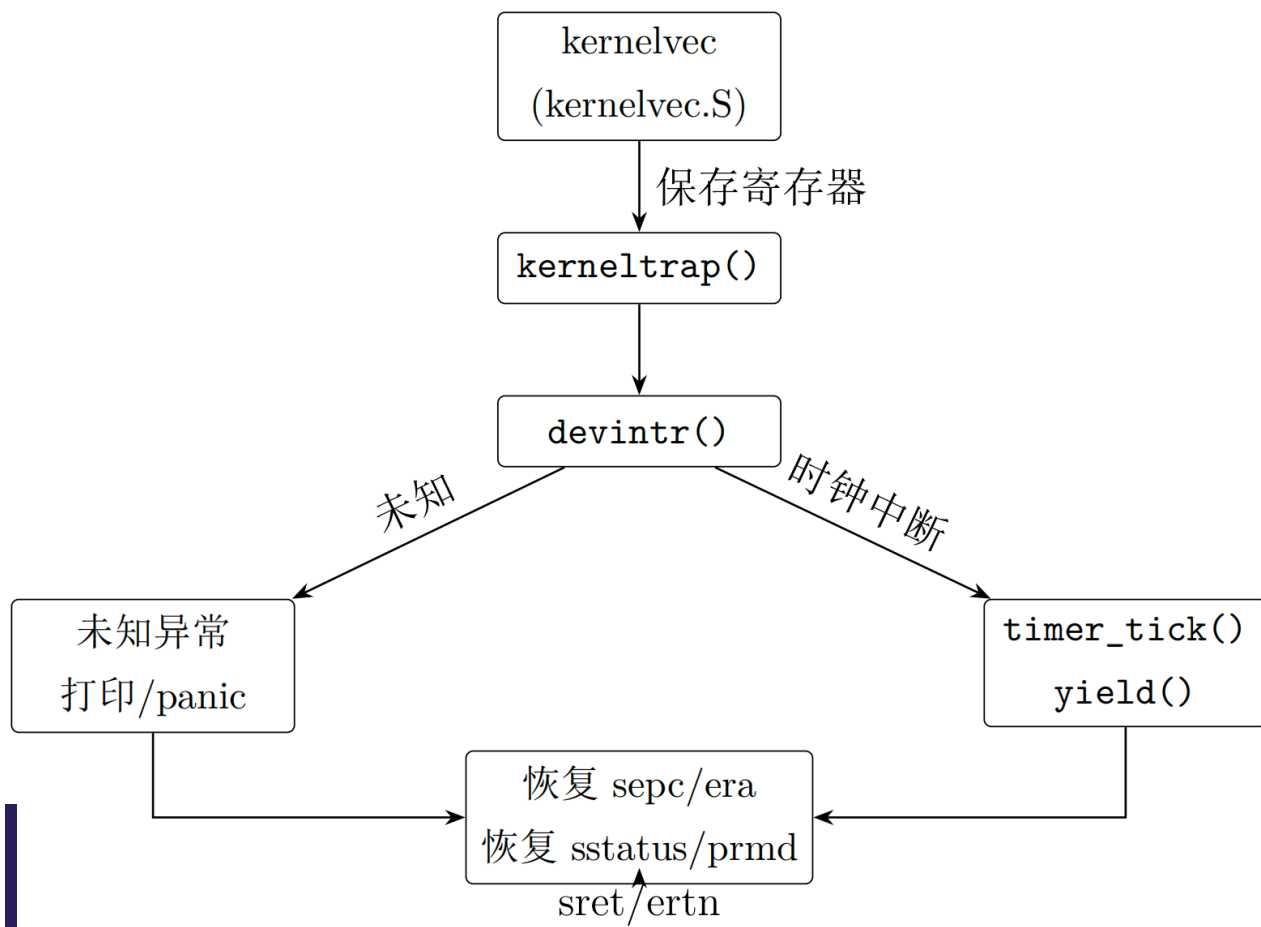
02 双架构启动链

- RISC-V 完成特权级切换;
- LoongArch 配置直映窗口与 TLB 环境;
- 最终统一进入内核入口。

硬件抽象与异常处理



武汉大学
WUHAN UNIVERSITY



03 Trap 与上下文分工

- trapframe 保存完整用户现场;
- context 保存最小调度现场;
- 用户态与内核态路径职责清晰。

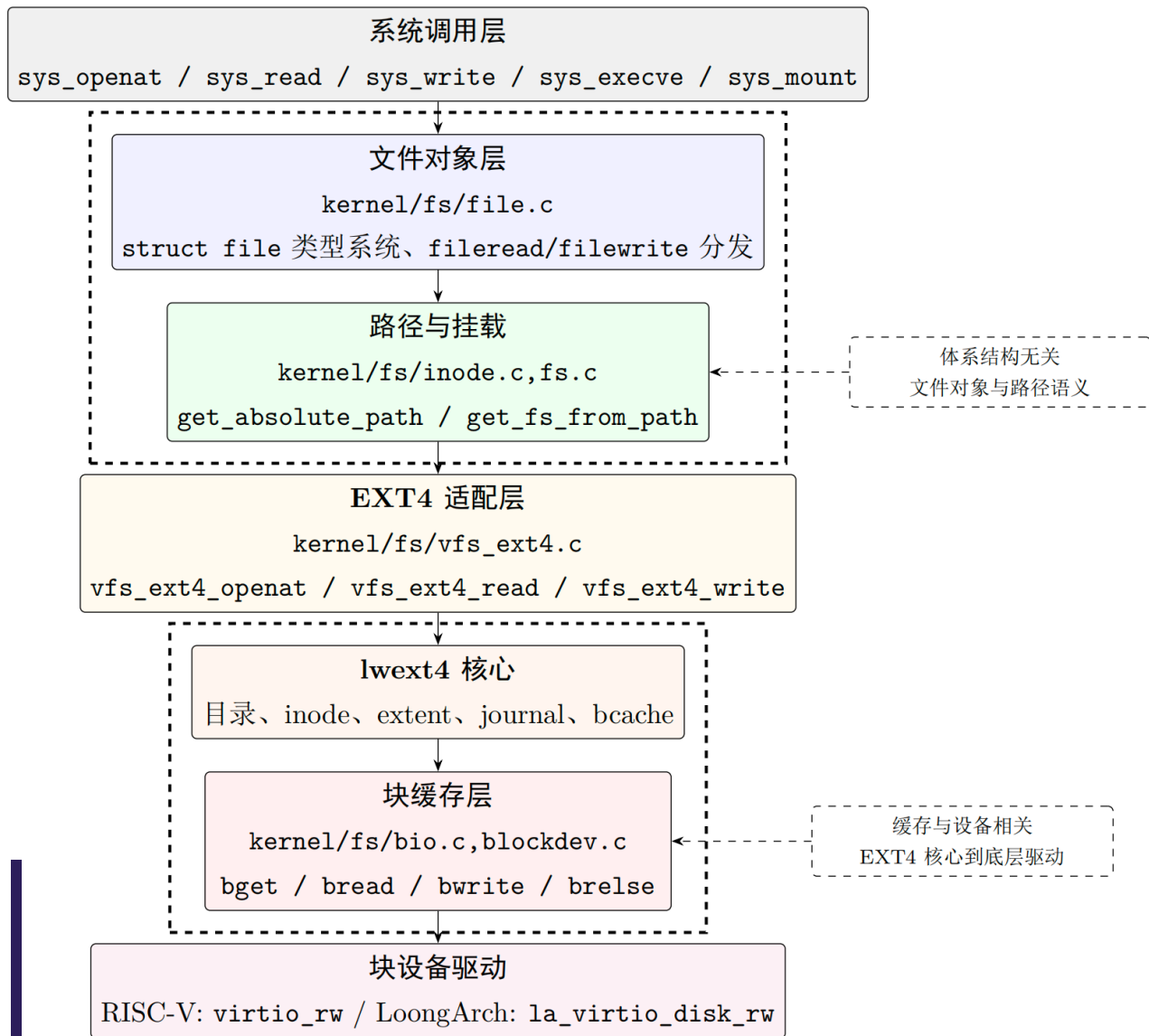
04 统一异常返回机制

- usertrap/kerneltrap 统一处理 syscall、缺页和中断;
- hsai_usertrapret 屏蔽 sret/ertn 差异;
- LoongArch 补充软件 TLB Refill。

文件系统



武汉大学
WUHAN UNIVERSITY



01 分层文件系统栈

- 系统调用层;
- 文件对象层;
- 路径与挂载层;
- EXT4、块缓存与设备驱动层。

02 统一文件对象与路径

- `struct file` 抽象普通文件、目录、管道、设备;
- 支持路径归一化;
- 支持最长前缀挂载匹配。

03 存储访问主链

- EXT4 适配层对接 `lwext4`;
- 块缓存提供 LRU 复用;
- 底层通过 VirtIO 驱动完成块 I/O。

04 兼容与装载能力

- 补齐 `/proc`、`/dev/null`、`/dev/shm` 等虚拟路径;
- `execve` 支持 ELF、PIE、动态链接器与脚本执行。



03

决赛内核优化

CAgent负载下的 内核基础性能优化

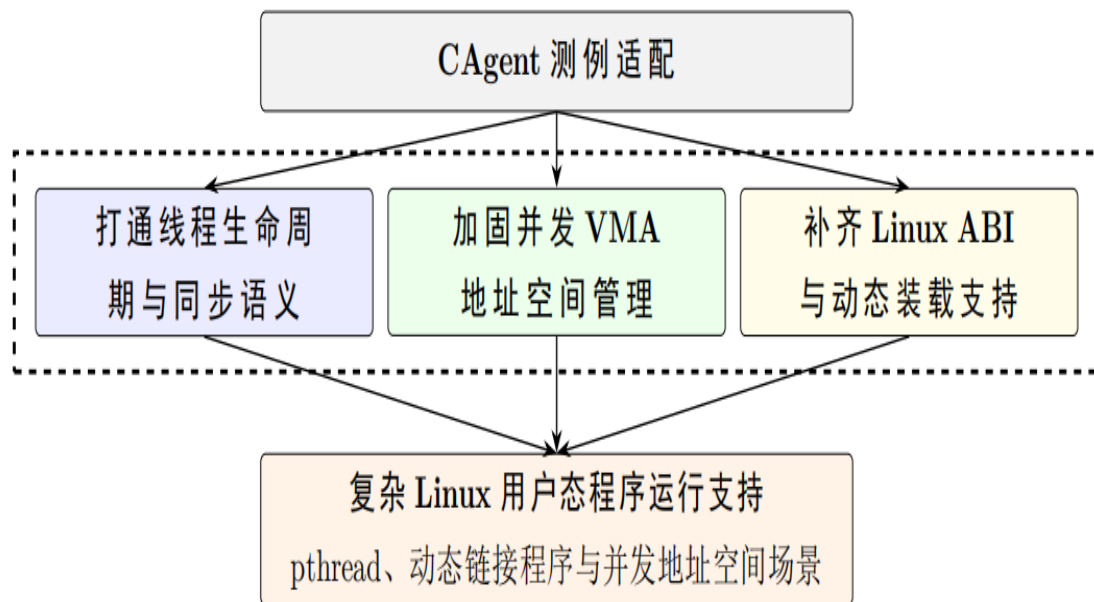


图 8.1: CAgent 测例驱动的内核优化方向

01 线程/进程生命周期修复

- 统一clone、TLS与TID管理;
- futex支撑等待与线程同步;
- 信号唤醒阻塞线程;
- 延迟回收线程栈与TrapFrame;

02 强化地址空间并发一致性

- VMA空隙选址与区间操作;
- vma_lock保护映射并发;
- 统一维护映射与页面权限;
- 修复pthread并发链表损坏;

03 用户态接口与程序加载

- 修复Linux结构;
- 解析软链接并加载解释器;
- 装载PT_LOAD与BSS;
- 完善用户栈与高地址空间;



复杂软件构建负载下的 内核性能优化

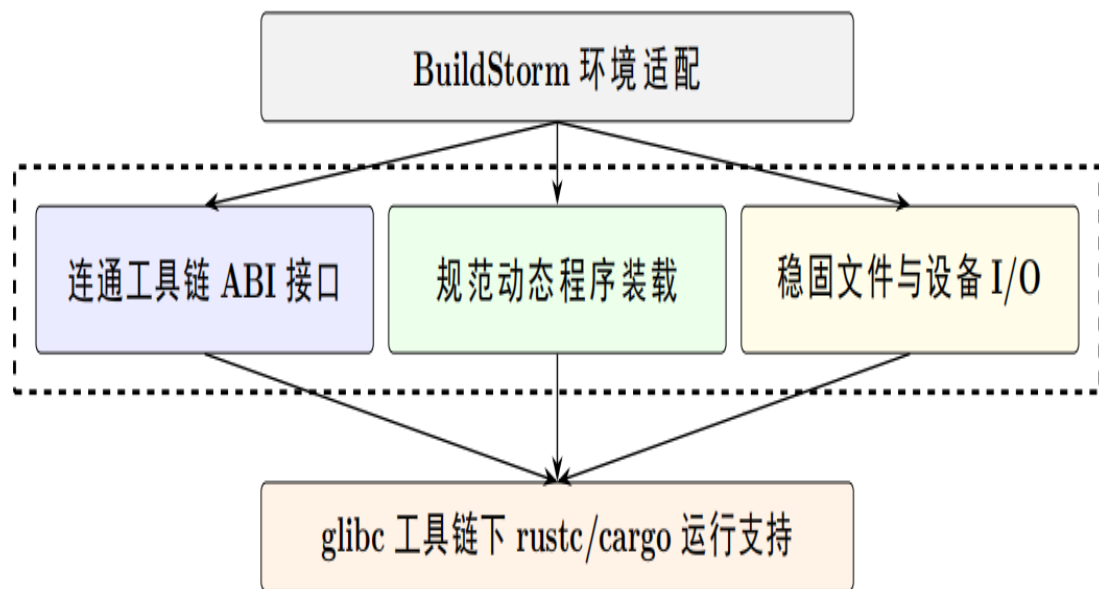


图 8.2: BuildStorm 测例驱动的内核优化方向

01 工具链接口兼容

- 修正TCGETS结构长度;
- 补齐glibc常用接口;
- 保留动态库搜索环境;
- 规范设备结点返回语义;

02 动态装载与地址空间维护

- 解析动态库版本软链接;
- 完善ELF解释器加载;
- 扩展高地址映射范围;
- 显式维护页面地址访问权限;

03 文件读取与设备访问漏洞修复

- 修复ext4大文件读取;
- 正确处理稀疏文件空洞;
- 隔离 VirtIO DMA缓冲区;



武汉大学
WUHAN UNIVERSITY

THANKS!

